



Semestrální práce z předmětu
Programování v jazyce ANSI C

Identifikace spamu naivním bayesovským klasifikátorem

4. ledna 2023

Autor:

Václav Tran

A21B0299P

`nuva@students.zcu.cz`

Vyučující:

Ing. Kamil Ekštein, Ph.D.

`kekstein@kiv.zcu.cz`

Obsah

1	Zadání	2
2	Analýza úlohy	3
2.1	Slovník	3
2.2	Naivní bayesovský klasifikátor	3
3	Popis implementace	5
3.1	Hash tabulka	5
3.1.1	Datové struktury	5
3.1.2	Vkládání prvků	6
3.1.3	Udržení složitosti	6
3.2	Naivní bayesovský klasifikátor	6
3.2.1	Datové struktury	6
3.2.2	Načítání textových souborů	7
3.2.3	Vytváření trénovací množiny	7
3.2.4	Fáze učení a fáze klasifikace	7
4	Uživatelská příručka	9
4.1	Přeložení a sestavení programu	9
4.2	Použití programu	10
5	Závěr	11
	Literatura	12

Kapitola 1

Zadání

Naprogramujte v ANSI C přenositelnou **konzolovou aplikaci**, která bude **rozhodovat, zda úsek textu** (textový soubor předaný jako parametr na příkazové řádce) **je nebo není spam**.

Program bude přijímat z příkazové řádky celkem **sedm** parametrů: První dva parametry budou vzor jména a počet trénovacích souborů obsahujících nevyžádané zprávy (tzv. **spam**). Třetí a čtvrtý parametr budou vzor jména a počet trénovacích souborů obsahujících vyžádané zprávy (tzv. **ham**). Pátý a šestý parametr budou vzor jména a počet testovacích souborů. Sedmý parametr představuje jméno výstupního textového souboru, který bude po dokončení činnosti Vašeho programu obsahovat výsledky klasifikace testovacích souborů.

Podrobnější popis viz [1].

Kapitola 2

Analýza úlohy

2.1 Slovník

Ke klasifikaci textového souboru podle zadání viz 1 se nejdříve musí vytvořit vhodná datová struktura, která bude uchovávat všechna slova textového souboru (slovník). Dvě takovéto datové struktury jsou trie a hash tabulka (tabulka s rozptýlenými položkami).

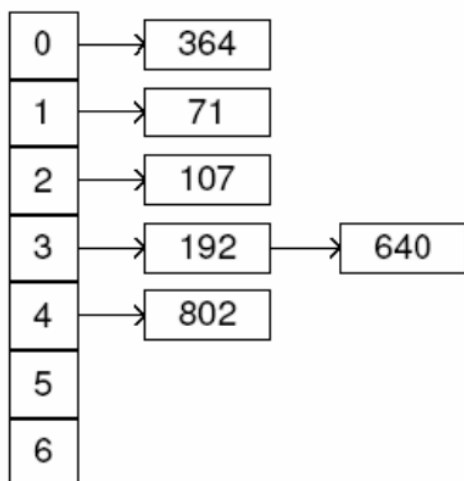
Obě datové struktury jsou poměrně porovnatelné. Amortizovaný čas na vyhledání slova v hash tabulce je $O(k)$, kde k je délka slova (za předpokladu, že byla vybraná vhodná hashovací funkce). Čas na vyhledání slova v trii je také $O(k)$. Výhoda využití trie je to, že nevyžaduje hashovací funkci. V případě, že hash tabulka bude využívat nevhodnou hashovací funkci, trie by byla vhodnější výběr.

V této úloze bude vybrána hash tabulka z důvodu větší znalosti implementace a využití. Na konci této kapitoly jsou uvedeny grafické podoby těchto datových struktur (převzato z [2]).

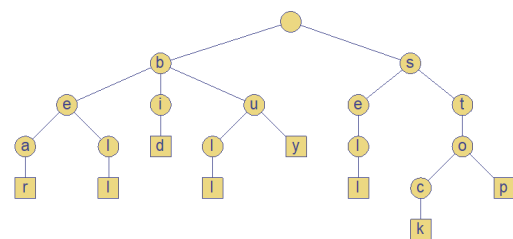
2.2 Naivní bayesovský klasifikátor

Ke klasifikaci bude využit algoritmus popsáný v podrobnějším zadání semestrální práce viz [1]. Tento algoritmus bude vyžadovat počet výskytů slova (frekvenci) a vypočítání pravděpodobností $P(c)$ a $P(\langle \text{word} | c \rangle)$, kde c je klasifikační třída. V této úloze jsou 2 třídy: *spam* a *ham* (pravděpodobnosti jsou podrobněji popsány viz [1]).

Do hash tabulky si v tomto případě nebudeme jenom ukládat slova, ale i jejich frekvenci a pravděpodobnost, že se dané slovo vyskytne v textovém souboru za podmínky klasifikace do třídy c . Do hash tabulky si budeme ukládat prvky typu datové struktury *node*, která v sobě bude uchovávat potřebné informace.



Obrázek 2.1: Příklad hash tabulky metodou rozptýlených indexů



Obrázek 2.2: Příklad standardní trie

Kapitola 3

Popis implementace

3.1 Hash tabulka

3.1.1 Datové struktury

Hash tabulka reprezentuje slovník. Využívá metodu rozptýlených indexů (zřetězení) a hashovací funkci "djb2" viz [3]. Do tabulky se budou ukládat prvky typu *node*, který v sobě uchovává

- *key* - název slova
- *freq* - počet výskytů slova
- *p_spam* - pravděpodobnost výskytu slova za podmínky klasifikace do třídy *spam*
- *p_ham* - pravděpodobnost výskytu slova za podmínky klasifikace do třídy *ham*
- *next* - ukazatel na další prvek typu *node*

Samotná hash tabulka je reprezentována datovou strukturou *hashtable*. Tato datová struktura v sobě uchovává

- *arr* - ukazatel na ukazatele na *node*
- *kapacita* - kapacita tabulky
- *count* - počet prvků v tabulce
- *uq_item_count* - počet unikátních prvků

Při vkládání prvků se vypočítá přes hashovací funkci **kyblík** (index), do kterého prvek bude patřit.

3.1.2 Vkládání prvků

V tomto kyblíku se vkládaný prvek bude porovnávat s každým prvkem v tomto kyblíku¹, aby se zjistilo, zda tento prvek už v tomto kyblíku není.

Pokud jsou si prvky rovny, změní se jenom počet výskytů slova. V opačném případě se prvek vloží na konci zřetězení a zvýší se počet unikátních prvků v tabulce. Při každém vkládání prvku (bez ohledu na to, zda prvek v této tabulce již existuje) se navýší počet prvků v tabulce *count*.

3.1.3 Udržení složitosti

Aby se udržela amortizovaná složitost $O(k)$ během vkládání a vyhledávání prvku, musí se zajistit, aby n/m byla konstanta z , kde n je počet prvků v tabulce a m je kapacita nebo počet kyblíků v tabulce.

Pokud n/m přesáhne hodnotu z , vytvoří se nový *arr* s dvakrát větší kapacitou než původní *arr* a všechny prvky (ukazatelé) původního *arr* se přesunou do nového *arr*. Je zřejmé, že se v tomto případě navýší i *kapacita* tabulky.

Toto zajišťuje funkce `rehash()`, která se vykonává při vkládání prvku za podmínek uvedených výše.

3.2 Naivní bayesovský klasifikátor

3.2.1 Datové struktury

Algoritmus pro tento klasifikátor vyžaduje definici množiny a trénovací množiny. K tomuto účelu byla vytvořena datová struktura *set*, která v sobě uchovává:

- *dict* - slovník (hash tabulku)
- *dict_file_count* - počet textových souborů, které byly načteny do slovníku
- *probability* - apriorní pravděpodobnost této množiny

Pro tento klasifikátor si připravíme 3 množiny:

1. *spam* - obsahuje prvky, patřící do třídy *spam*
2. *ham* - obsahuje prvky, patřící do třídy *ham*
3. *total* - obsahuje prvky, patřící do třídy *spam* nebo *ham*

¹Prvky jsou ekvivalentní pokud obsahují stejné slovo

Účely těchto množin je vysvětleno níže v sekci 3.2.3.

Tyto tři množiny se budou uchovávat v datové struktuře *trainset*, která reprezentuje trénovací množinu. Uchovává v sobě 3 výše uvedené množiny a počet množin v trénovací množině.

3.2.2 Načítání textových souborů

Každý soubor musí být zpracovaný a extrahovaná slova ze souboru musí být uložena do daného slovníku. Toto nám zajišťuje funkce `load_words()`.

Funkce `load_words()` načte ale jenom jeden soubor s daným názvem. Pro naše účely je toto nedostatečné, potřebujeme načíst všechny soubory s názvem: *vzorN*, kde *N* je celé číslo z intervalu $\langle 1, N \rangle$ a všechny názvy souborů obsahují příponu ".txt". Tuto funkčnost nám zajišťují funkce `load_words_from_vzor_files()` a `create_vzor_name()`.

Funkce `create_vzor_name()` vytvoří podle daného řetězce *vzor* a celého čísla *I* řetězec *vzorI*. Funkce `load_words_from_vzor_files()` pomocí funkce `load_words()` a funkce `create_vzor_name()` má všechny potřebné prvky k tomu, aby načetla všechny slova ze souborů *vzorN.txt* do daného slovníku.

3.2.3 Vytváření trénovací množiny

Vytváření trénovací množiny nám zajišťuje funkce `create_training_set()`. Pomocí funkcí z 3.2.2 se naplní tyto 3 vytvořené množiny: *spam*, *ham* a *total*. V trénovací množině budeme přistupovat k těmto množinám pomocí indexů 0, 1, 2. Index reprezentující *spam* bude 0, index reprezentující *ham* bude 1 a index reprezentující *total* bude 3.

U této trénovací množiny ale nastavíme počet prvků v této množině na 2. Důvodem je, že při fázi klasifikace a učení budeme chtít jenom procházet množiny *spam* a *ham*. K množině *total* budeme přistupovat manuálně. Důsledkem této vychytávky je, že při uvolňování trénovací množiny pomocí funkce `free_trainset()` se k počtu prvků v trénovací množině musí přičíst 1, aby se uvolnila i množina *total*.

3.2.4 Fáze učení a fáze klasifikace

Fáze učení je reprezentována funkcí `NB_learn_text()` a fáze klasifikace jednoho textového souboru je reprezentována funkcí `NB_classify_text()`. Fáze klasifikace pro více souborů je reprezentována metodou `NB_classify_vzor_text()`.

Funkce `NB_learn_text()` a `NB_classify_text()` byly vytvořeny podle pseudokódu viz [1].

Funkce `NB_classify_vzor_text()` využívá funkci `create_vzor_name()` k vytvoření vyžadovaných názvů souborů. Všechny tyto textové soubory se klasifikují a vytvoří se výstupní soubor s názvem předaný jako parametr.

Tento výstupní soubor bude obsahovat klasifikaci jednotlivých souborů ve formátu uvedeném v [1]. Ukázka výstupního souboru je také viditelná viz 4.2.

Kapitola 4

Uživatelská příručka

4.1 Přeložení a sestavení programu

Pro přeložení a sestavení programu byly vytvořeny 2 `makefile` soubory.

`makefile.win` je určen pro Windows a `makefile` je určen pro Linux. Uživatelé na Windows si navíc budou muset nainstalovat kompilátor jazyka C (např. **MinGW** nebo **MSVC**). Příkaz pro sestavení programu může vypadat takto:

Windows¹:

```
mingw32-make -f makefile.win
```

Linux:

```
make -f makefile
```

Po zadání vybraného příkazu se vytvoří soubor `spamid.exe`.

¹Zde záleží, jaký kompilátor využíváte. V této ukázce se využívá MinGW.

4.2 Použití programu

Program by se měl spouštět příkazem:

```
spamid.exe <spam><spam-cnt><ham><ham-cnt><test><test-cnt><out-file>
```

Symbole `<spam>`, `<ham>` a `<test>` představují vzory jména vstupních souborů. Symbole `<spam-cnt>`, `<ham-cnt>`, `<test-cnt>` představují počty vstupních souborů.

Symbole `<spam>`, `<ham>` určují soubory, ze kterých bude probíhat fáze učení. Symbol `<test>` určuje soubory, které se budou klasifikovat do třídy *spam* nebo *ham*. Symbol `<out-file>` zde určuje název výstupního souboru, kde budou uloženy výsledky klasifikace jednotlivých textových souborů.

Soubory klasifikované do třídy *spam* představují nevyžádané zprávy. Soubory klasifikované do třídy *ham* jsou naopak zprávy, které přijímáme.

Všechny vstupní soubory musí být umístěny v podadresáři `data`. Obsah vstupního souboru by mohl vypadat takto:

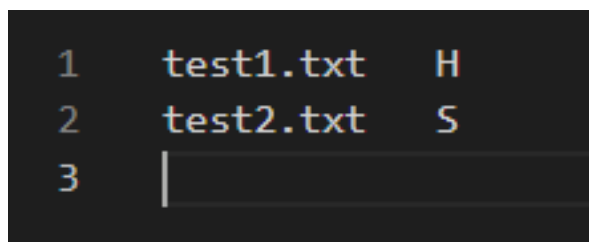
```
hello money rich quick research center content length
```

Slova musí být od sebe oddělena právě jednou mezerou.

Vstupní soubory mají následující pojmenování: `vzorN`, kde `N` je celé číslo z intervalu $\langle 1; N \rangle$. Přípona všech vstupních souborů je `.txt`, přípona není součástí vzoru. Váš program tedy může být během testování spuštěn například takto:

```
spamid.exe spam 300 ham 300 test 2 result.txt
```

Po vykonání příkazu uvedeného výše může soubor `result.txt` vypadat takto:



```
1 test1.txt H
2 test2.txt S
3 |
```

Obrázek 4.1: Soubor `result.txt` po vykonání příkazu

Kapitola 5

Závěr

Program správně klasifikuje textové soubory s $> 90\%$ úspěšností nad testovacími daty. Zhodnotil bych tudíž splnění zadání jako úspěšné.

Pro správnou klasifikaci při odevzdávání této práce přes validátor se s apriorní pravděpodobností nepočítalo z důvodu nízké správnosti výsledků.

Jeden z možných problémů je malá "obecnost" u funkce `create_trainingset()` a funkce `free_trainingset` viz 3.2.3.

Literatura

- [1] Kamil Ekštein. ZADÁNÍ SEMESTRÁLNÍ PRÁCE - IDENTIFIKACE SPAMU NAIVNÍM BAYESOVSKÝM KLASIFIKÁTOREM. <https://www.kiv.zcu.cz/studies/predmety/pc/data/works/sw2022-02.pdf>, 2022. [Online].
- [2] Pavel Mautner. KIV-PT Přednášky. <https://courseware.zcu.cz/portal/studium/courseware/kiv/pt/prednasky.html>. [Online].
- [3] Dan Bernstein. djb2. <http://www.cse.yorku.ca/~oz/hash.html>. [Online].